6 Introducing Operator Overloading

Chapter Objectives

6.1 The basics of operator overloading

6.2 Overloading binary operators

6.3 Overloading the relational and logical operators

6.4 Overloading a unary operator

6.5 Using friend operator functions

6.6 A closer look at the assignment operator

6.7 Overloading the [ ] subscript operator

THIS chapter introduces another important C++ feature: operator overloading. This feature allows you to define the meaning of the C++ operators relative to classes that you define. By overloading operators, you can seamlessly add new data types to you program.

Review Skills Check

Before proceeding, you should be able to correctly answer the following questions and do the exercises.

1. Show how to overload the constructor for the following class so that uninitialized objects can also be created. (When creating uninitialized objects, give **x** and **y** the value 0.)

```
class myclass
{
    int x, y;
public:
    myclass(int i, int j) { x=i; y=j; }
    //
};
```

2. Using the class from Question 1, show how you can avoid overloading **myclass()** by using default arguments.

3. What is wrong with these two overloaded functions?

```
void f(int a);
void f(int &a);
```

4. When is it appropriate to use default arguments? When is it probably a bad idea?

5. Given the following class definition, is it possible to dynamically allocate an array of these objects?

```
class test
{
    char *p;
    int *q;
    int count;
public:
    test(char *x, int *y, int c)
    {
        p = x;
        q = y;
        count = c;
    }
    // ...
};
```

6. What is a copy constructor and under what circumstances is it called?

6.1 THE BASICS OF OPERATOR OVERLOADING

Operator overloading resembles function overloading. In fact, operator overloading is really just a type of function overloading. However, some additional rules apply. For example, an operator

is always overloaded relative to a user-defined type, such as a class. Other differences will be discussed as needed.

When an operator is overloaded, that operator loses none of its original meaning. Instead, it gains additional meaning relative to the class for which it is defined.

To overload an operator, you create an *operator function*. Most often an operator function is a member or a friend of the class for which it is defined. However, there is a slight difference between a member operator function and a friend operator function. The first part of this chapter discusses the creation of member operator functions. Then friend operator functions are discussed.

The general form of a member operator function is shown here:

```
return_type class_name::operator#(arg_list)
{
    // operation to be performed
}
```

The return type of an operator function is often the class for which it is defined. (However, an operator function is free to return any type.) The operator being overloaded is substituted for the #. For example, if the + is being overloaded, the function name would be **operator +**. The contents of *arg-list* vary depending upon how the operator function is implemented and the type of operator being overloaded.

There are two important restrictions to remember when you are overloading an operator. First, the precedence of the operator cannot be changed. Second, the number of operands that an operator takes cannot be altered. For example, you cannot overload the / operator so that it takes only one operand.

Most C++ operators can be overloaded. The only operators that you cannot overload are shown here:

. :: .* ?

Also, you cannot overload the preprocessor operators. (The **.*** operator is highly specialized and is beyond the scope of this book.)

Remember that C++ defines operators very broadly, including such things as the [ ] subscript operators, the **( )** function call operators, **new** and **delete**, and the **.** (dot) and **->** (arrow) operators. However, this chapter concentrates on overloading the most commonly used operators.

Except for the =, operator functions are inherited by any derived class. However, a derived class is free to overload any operator it chooses (including those overloaded by the base class) relative to itself.

You have been using two overloaded operators: << and >>. These operators have been overloaded to perform console I/O. As mentioned, overloading these operators to perform I/O does not prevent them from performing their traditional jobs of left shift and right shift.

While it is permissible for you to have an operator function perform *any* activity-whether related to the traditional use of the operator or not-it is best to have an overloaded operator's actions stay within the spirit of the operator's traditional use. When you create overloaded operators that stray from this principal, you run the risk of substantially destructuring your program. For example, overloading the / so that the phrase "I like C++" is written to a disk file 300 times is a fundamentally confusing misuse of operator overloading!

The preceding paragraph notwithstanding, there will be times when you need to use an operator in a way not related to its traditional usage. The two best examples of this are the << and >> operators, which are overloaded for console I/O. However, even in these cases, the left and right arrows provide a visual "clue" to their meaning. Therefore, if you need to overload an operator in a nonstandard way, make the greatest effort possible to use an appropriate operator.

One final point: operator functions cannot have default arguments.

6.2 OVERLOADING BINARY OPERATORS

When a member operator function overloads a binary operator, the function will have only one parameter. This parameter will receive the object that is on the right side of the operator. The object on the left side is the object that generates the call to the operator function and is passed implicitly by **this**.

It is important to understand that operator functions can be written with many variations. The examples here and elsewhere in this chapter are not exhaustive, but they do illustrate several of the most common techniques.

EXAMPLES

1. The following program overloads the + operator relative to the **coord** class. This class is used to maintain X, Y coordinates.

```
// Overload the + relative to coord class
#include <iostream>
using namespace std;

class coord
{
    int x, y; // coordinate values
public:
    coord() { x=0; y=0; };
    coord(int i, int j) { x=i; y=j; }
    void get_xy(int &i, int &j) { i=x; j=y; }
    coord operator+(coord ob2);
};

// Overload + relative to coord class.
coord coord::operator+(coord ob2)
{
    coord temp;
    temp.x = x + ob2.x;
    temp.y = y + ob2.y;

    return temp;
}

int main()
{
    coord o1(10, 10), o2(5, 3), o3;
    int x, y;

    o3 = o1 + o2; // add two objects - this calls operator+

    o3.get_xy(x, y);
    cout << "(o1+o2) X: " << x << ", Y: " << y << "\n";

    return 0;
}
```

This program displays the following:

```
(o1+o2) X: 15, Y: 13
```

Let's look closely at this program. The **operator+()** function returns an object of type **coord** that has the sum of each operand's X coordinates in **x** and the sum of the Y coordinates in **y**. Notice that a temporary object called **temp** is used inside **operator+()** to hold the result, and it is this object that is returned. Notice also that neither operand is modified. The reason for **temp** is easy to understand. In this situation (as in most), the + has been overloaded in a manner consistent with its normal arithmetic use. Therefor, it was important that neither operand be changed. For example, when you add 10+4, the result is 14, but neither the 10 nor the 4 is modified. Thus, a temporary object is needed to hold the result.

The reason that the **operator+()** function returns an object of type **coord** is that it allows the result of the addition of **coord** objects to be used in larger expressions. For example, the statement

```
o3 = o1 + o2
```

is valid only because the result of **o1 + o2** is a **coord** object that can be assigned to **o3**. If a different type had been returned, this statement would have been invalid. Further, by returning a **coord** object, the addition operator allows a string of additions. For example, this is a valid statement:

```
o3 = o1 + o2 + o1 + o3;
```

Although there will be situations in which you want an operator function to return something other than an object for which it is defined, most of the time operator functions that you create will return an object of their class. (The major exception to this rule is when the relational and logical operators are overloaded. This situation is examined in Section 6.3, "Overloading the Relational and Logical Operators," later in this chapter.)

One final point about this example. Because a **coord** object is returned, the following statement is also perfectly valid:

```
(o1+o2).get_xy(x, y);
```

Here the temporary object returned by **operator+()** is used directly. Of course, after this statement has executed, the temporary object is destroyed.

2. The following version of the preceding program overloads the **-** and the **=** operators relative to the **coord** class.

```
// Overload the +, -, and = relative to coord class.
#include <iostream>
using namespace std;

class coord
{
    int x, y; // coordinate values
public:
    coord() { x=0; y=0; };
    coord(int i, int j) { x=i; y=j; }
    void get_xy(int &i, int &j) { i=x; j=y; }
    coord operator+(coord ob2);
    coord operator-(coord ob2);
    coord operator=(coord ob2);
};
```

```
// Overload + relative to coord class.
coord coord::operator+(coord ob2)
{
    coord temp;
    temp.x = x + ob2.x;
    temp.y = y + ob2.y;

    return temp;
}

// Overload - relative to coord class.
coord coord::operator-(coord ob2)
{
    coord temp;
    temp.x = x - ob2.x;
    temp.y = y - ob2.y;

    return temp;
}

// Overload = relative to coord class.
coord coord::operator=(coord ob2)
{
    x = ob2.x;
    y = ob2.y;

    return *this; // return the object that is assigned
}

int main()
{
    coord o1(10, 10), o2(5, 3), o3;
    int x, y;

    o3 = o1 + o2; // add two objects - this calls operator+
    o3.get_xy(x, y);
    cout << "(o1+o2) X: " << x << ", Y: " << y << "\n";

    o3 = o1 - o2; // subtract two objects
    o3.get_xy(x, y);
    cout << "(o1-o2) X: " << x << ", Y: " << y << "\n";

    o3 = o1; // assign an object
    o3.get_xy(x, y);
    cout << "(o3=o1) X: " << x << ", Y: " << y << "\n";

    return 0;
}
```

The **operator-()** function is implemented similarly to **operator+()**. However, the above example illustrates a crucial point when you are overloading an operator in which the order

of the operands is important. When the **operator+()** function was created, it did not matter which order the operands were in. (That is, A + B is the same as B + A.) However, the subtraction operation is order dependent. Therefore, to subtract the operand on the right from the operand on the left. Because it is the left operand that generates the call to **operator-()**, the subtraction must be in this order:

```
x - ob2.x;
```

Remember: *When a binary operator is overloaded, the left operand is passed implicitly to the function and the right operand is passed as an argument.*

Now look at the assignment operator function. The first thing you should notice that the left operand (that is, the object being assigned a value) is modified by the operation. This is in keeping with the normal meaning of assignment. The second thing to notice is that the function returns ***this**. That is, the **operator=()** function returns the object that is being assigned to. The reason for this is to allow a series of assignments to be made. As you should know, in C++, the following type of statement is syntactically correct (and indeed very common):

```
a = b = c = d = 0;
```

By returning ***this**, the overloaded assignment operator allows objects of type **coord** to be used in a similar fashion. For example, this is perfectly valid:

```
o3 = o2 = o1;
```

Keep in mind that there is no rule that requires an overloaded assignment function to return the object that receives the assignment. However, if you want the overloaded = to behave relative to its class the way it does for the built-in types, it must return ***this**.

3. It is possible to overload an operator relative to a class so that the operand on the right side is an object of a built-in type, such as an integer, instead of the class for which the operator function is a member. For example, here the + operator is overloaded to add an integer value to a **coord** object:

```
// Overload the + for ob + int as well as ob + ob.
#include <iostream>
using namespace std;

class coord
{
    int x, y; // coordinate values
public:
    coord() { x=0; y=0; };
    coord(int i, int j) { x=i; y=j; }
    void get_xy(int &i, int &j) { i=x; j=y; }
    coord operator+(coord ob2); // ob + ob
    coord operator+(int i); // ob + int
};

// Overload + relative to coord class.
coord coord::operator+(coord ob2)
{
    coord temp;
    temp.x = x + ob2.x;
    temp.y = y + ob2.y;
```

```
    return temp;
}

// Overload + for ob + int
coord coord::operator+(int i)
{
    coord temp;
    temp.x = x + i;
    temp.y = y + i;

    return temp;
}

int main()
{
    coord o1(10, 10), o2(5, 3), o3;
    int x, y;

    o3 = o1 + o2; // add two objects - calls operator+(coord)
    o3.get_xy(x, y);
    cout << "(o1+o2) X: " << x << ", Y: " << y << "\n";

    o3 = o1 + 100; // add object + int - calls operator+(int)
    o3.get_xy(x, y);
    cout << "(o1+100) X: " << x << ", Y: " << y << "\n";

    return 0;
}
```

It is important to remember that when you are overloading a member operator function so that an object can be used in an operation involving a built-in type, the built-in type must be on the right side of the operator. The reason for this is easy to understand: It is the object on the left that generates the call to the operator function. For instance, what happens when the compiler sees the following statement?

```
o3 = 19 + o1; // int + ob
```

There is no built-in operation defined to handle the addition of an integer to an object. The overloaded **operator+(int i)** function works only when the object is on the left. Therefore, this statement generates a compile-time error. (Soon you will see one way around this restriction.)

4. You can use a reference parameter in an operator function. For example, this is a perfectly acceptable way to overload the + operator relative to the **coord** class:

```
// Overload + relative to coord class using references.
coord coord::operator+(coord &ob2)
{
    coord temp;

    temp.x = x + ob2.x;
    temp.y = y + ob2.y;
```

```
    return temp;
}
```

One reason for using a reference parameter in an operator function is efficiency. Passing objects as parameters to functions often incurs a large amount of overhead and consumes a significant number of CPU cycles. However, passing the address of an object is always quick and efficient. If the operator is going to be used often, using a reference parameter will generally improve performance significantly.

Another reason for using a reference parameter is to avoid the trouble caused when a copy of an operand is destroyed. As you know from previous chapters, when an argument is passed by value, a copy of that argument is made. If that object has a destructor function, when the function terminates, the copy's destructor is called. In some cases it is possible for the destructor to destroy something needed by the calling object. If this is the case, using a reference parameter instead of a value parameter is an easy (and efficient) way around the problem. Of course, you could also define a copy constructor that would prevent this problem in the general case.

EXERCISES

1. Relative to **coord**, overload the * and / operators. Demonstrate that they work.

2. Why would the following be an inappropriate use of an overloaded operator?

```
coord coord::operator%(coord ob)
{
    double i;

    cout << "Enter a number: ";
    cin >> i;
    cout << "root of " << i << " is ";
    cout << sqrt(i);
}
```

3. On your own, experiment by changing the return types of the operator functions to something other than **coord**. See what types of errors result.

6.3 OVERLOADING THE RELATIONAL AND LOGICAL OPERATORS

It is possible to overload the relational and logical operators. When you overload the relational and logical operators so that they behave in their traditional manner, you will not want the operator functions to return an object of the class for which they are defined. Instead, they will return an integer that indicates either true or false. This not only allows these operator functions to return a true/false value, it also allows the operators to be integrated into larger relational and logical expressions that involve other types of data.

Note: *If you are using a modern C++ compiler, you can also have an overloaded relational or logical operator function return a value of type* ***bool****, although there is no advantage to doing so. As explained in Chapter 1, the* ***bool*** *type defines only two values:* ***true*** *and* ***false****. These values are automatically converted into nonzero and 0 values. Integer nonzero and 0 values are automatically converted into* ***true*** *and* ***false****.*

EXAMPLE

1. In the following program, the == and && operators are overloaded:

```
// Overload the == and && relative to coord class.
#include <iostream>
using namespace std;

class coord
{
    int x, y; // coordinate values
public:
    coord() { x=0; y=0; };
    coord(int i, int j) { x=i; y=j; }
    void get_xy(int &i, int &j) { i=x; j=y; }
    int operator==(coord ob2);
    int operator&&(coord ob2);
};

// Overload the == operator for coord.
int coord::operator==(coord ob2)
{
    return x==ob2.x && y==ob2.y;
}

// Overload the && operator for coord.
int coord::operator&&(coord ob2)
{
    return (x && ob2.x) && (y && ob2.y);
}

int main()
{
    coord o1(10, 10), o2(5, 3), o3(10, 10), o4(0, 0);

    if(o1==o2)
        cout << "o1 same as o2\n";
    else
        cout << "o1 and o2 differs\n";

    if(o1==o3)
        cout << "o1 same as o3\n";
    else
        cout << "o1 and o3 differ\n";

    if(o1&&o2)
        cout << "o1 && o2 is true\n";
    else
        cout << "o1 && o2 is false\n";

    if(o1&&o4)
        cout << "o1 && o4 is true\n";
    else
```

```
        cout << "o1 && o4 is false\n";

    return 0;
}
```

EXERCISE

1. Overload < and > operator relative to the **coord** class.

6.4 OVERLOADING A UNARY OPERATOR

Overloading a unary operator is similar to overloading a binary operator except that there is only one operand to deal with. When you overload a unary operator using a member function, the function has no parameters. Since there is only one operand, it is this operand that generates the call to the operator function. There is no need for another parameter.

EXAMPLES

1. The following program overloads the increment operator (++) relative to the **coord** class

```
// Overload ++ relative to coord class.
#include <iostream>
using namespace std;

class coord
{
    int x, y; // coordinate values
public:
    coord() { x=0; y=0; };
    coord(int i, int j) { x=i; y=j; }
    void get_xy(int &i, int &j) { i=x; j=y; }
    coord operator++();
};

// Overload ++ for coord.
coord coord::operator++()
{
    x++;
    y++;

    return *this;
}

int main()
{
    coord o1(10, 10);
    int x, y;

    ++o1; // increment an object
    o1.get_xy(x, y);
    cout << "(++o1) X: " << x << ", Y: " << y << "\n";
```

```
    return 0;
}
```

Since the increment operator is designed to increase its operand by 1, the overloaded ++ modifies the object it operates upon. The function also returns the object that it increments. This allows the increment operator to be used as part of a larger statement, such as this:

```
o2 = ++o1;
```

As with the binary operators, there is no rule that says you must overload a unary operator so that it reflects its normal meaning. However, most of the time this is what you will want to do.

2. In early version of C++, when an increment or decrement operator was overloaded, there was no way to determine whether an overloaded ++ or – preceded or followed its operand. That is, assuming the preceding program, these two statements would have been identical:

```
o1++;
++o1;
```

However, the modern specification for C++ has defined a way by which the compiler can distinguish between these two statements. To accomplish this, create two versions of the **operator++()** function. The first is defined as shown in the preceding example. The second is declared like this:

```
coord coord::operator++(int notused);
```

If the ++ operator precedes its operand, the **operator++()** function is called. However, if the ++ follows its operand, the **operator++(int notused)** function is used. In this case, **notused** will always be passed the value 0. Therefor, if the difference between prefix and postfix increment or decrement is important to your class objects, you will need to implement both operator functions.

3. As you know, the minus sign is both a binary and a unary operator in C++. You might be wondering how you can overload it so that it retains both of these uses relative to a class that you create. The solution is actually quite easy: you simply overload it twice, once as a binary operator and once as a unary operator. This program shows how:

```
// Overload the - relative to coord class.
#include <iostream>
using namespace std;

class coord
{
    int x, y; // coordinate values
public:
    coord() { x=0; y=0; };
    coord(int i, int j) { x=i; y=j; }
    void get_xy(int &i, int &j) { i=x; j=y; }
    coord operator-(coord ob2); // binary minus
    coord operator-(); // unary minus
};
```

```
// Overload - relative to coord class.
coord coord::operator-(coord ob2)
{
    coord temp;

    temp.x = x - ob2.x;
    temp.y = y - ob2.y;

    return temp;
}

// Overload unary - relative to coord class.
coord coord::operator-()
{
    x = -x;
    y = -y;
    return *this;
}

int main()
{
    coord o1(10, 10), o2(5, 7);
    int x, y;

    o1 = o1 - o2; // subtraction
    o1.get_xy(x, y);
    cout << "(o1-o2) X: " << x << ", Y: " << y << "\n";

    o1 = -o1; // negation
    o1.get_xy(x, y);
    cout << "(-o1) X: " << x << ", Y: " << y << "\n";

    return 0;
}
```

As you can see, when the minus is overloaded as a binary operator, it takes one parameter. When it is overloaded as a unary operator, it takes no parameter. This difference in the number of parameters is what makes it possible for the minus to be overloaded for both operations. As the program indicates., when the minus sign is used as a binary operator, the **operator-(coord ob2)** function is called. When it is used as a unary minus, the **operator-()** function is called.

EXERCISES

1. Overload the – operator for the **coord** class. Create both its prefix and postfix forms.

2. Overload the + operator for the **coord** class so that it is both a binary operator (as shown earlier) and a unary operator. When it is used as a unary operator, have the + make any negative coordinate value positive.

6.5 USING FRIEND OPERATOR FUNCTIONS

As mentioned at the start of this chapter, it is possible to overload an operator relative to a class by using a friend rather than a member function. As you know, a friend function does not have a **this** pointer. In the case of a binary operator, this means that a friend operator function is passed both operands explicitly. For unary operators, the single operand is passed. All other things being equal, there is no reason to use a friend rather than a member operator function, with one important exception, which is discussed in the examples.

Remember: *You cannot use a friend to overload the assignment operator. The assignment operator can be overloaded only by a member operator function.*

1. Here **operator+()** is overloaded for the **coord** class using a friend function:

```
// Overload the + relative to coord class using a friend.
#include <iostream>
using namespace std;

class coord
{
    int x, y; // coordinate values
public:
    coord() { x=0; y=0; };
    coord(int i, int j) { x=i; y=j; }
    void get_xy(int &i, int &j) { i=x; j=y; }
    friend coord operator+(coord ob1, coord ob2);
};

// Overload + using a friend.
coord operator+(coord ob1, coord ob2)
{
    coord temp;

    temp.x = ob1.x + ob2.x;
    temp.y = ob1.y + ob2.y;

    return temp;
}

int main()
{
    coord o1(10, 10), o2(5, 3), o3;
    int x, y;

    o3 = o1 + o2; // add two objects - this calls operator+()
    o3.get_xy(x, y);
    cout << "(o1+o2) X: " << x << ", Y: " << y << "\n";

    return 0;
}
```

Notice that the left operand is passed to the first parameter and the right operand is passed to the second parameter.

2. Overloading an operator by using a friend provides one very important feature that member functions do not. Using a friend operator function, you can allow objects to be used in operations involving built-in types in which the built-in type is on the left side of the operator. As you saw earlier in this chapter, it is possible to overload a binary member operator function such that the left operand is an object and the right operand is a built-in type. But it is not possible to use a member function to allow the built-in type to occur on the left side of the operator. For example, assuming an overload member operator function, the first statement shown here is legal; the second is not:

```
ob1 = ob2 + 10; // legal
ob1 = 10 + ob2; // illegal
```

While it is possible to organize such statements like the first, always having to make sure that the object is on the left side of the operand and the built-in type on the right can be cumbersome restriction. The solution to this problem is to make the overloaded operator functions friends and define both possible situations.

As you know, a friend operator function is explicitly passed *both* operands. Thus, it is possible to define one overloaded friend function so that the left operand is an object and the right operand is the other type. Then you could overload the operator again with the left operand being the built-in type and the right operand being the object. The following program illustrates this method:

```
// Use friend operator functions to add flexibility.
#include <iostream>
using namespace std;

class coord
{
    int x, y; // coordinate values
public:
    coord() { x=0; y=0; };
    coord(int i, int j) { x=i; y=j; }
    void get_xy(int &i, int &j) { i=x; j=y; }
    friend coord operator+(coord ob1, int i);
    friend coord operator+(int i, coord ob1);
};

// Overload + for ob + int
coord operator+(coord ob1, int i)
{
    coord temp;

    temp.x = ob1.x + i;
    temp.y = ob1.y + i;

    return temp;
}

// Overload + for int + ob
coord operator+(int i, coord ob1)
{
    coord temp;
```

```
    temp.x = ob1.x + i;
    temp.y = ob1.y + i;

    return temp;
}

int main()
{
    coord o1(10, 10);
    int x, y;

    o1 = o1 + 10; // object + integer
    o1.get_xy(x, y);
    cout << "(o1+10) X: " << x << ", Y: " << y << "\n";

    o1 = 99 + o1; // integer + object
    o1.get_xy(x, y);
    cout << "(99+o1) X: " << x << ", Y: " << y << "\n";

    return 0;
}
```

As a result of overloading friend operator functions for both situations, both of these statements are now valid:

```
o1 = o1 + 10;
o1 = 99 + o1;
```

3. If you want to use a friend operator function to overload either the ++ or – unary operator, you must pass the operand to the function as a reference parameter. This is because friend functions do not have **this** pointers. Remember that the increment and decrement operators imply that the operand will be modified. However, if you overload these operators by using a friend that uses a value parameter, any modifications that occur to the parameter inside the friend operator function will not affect the object that generated the call. And since no pointer to the object is passed implicitly (that is, there is no **this** pointer) when a friend function is used, there is no way for the increment or decrement to affect the operand.

   However, if you pass the operand to the friend as a reference parameter, changes that occur inside the friend function affect the object that generates the call. For example, here is a program that overloads the ++ operator by using a friend function:

```
// Overload the ++ using a friend.
#include <iostream>
using namespace std;

class coord
{
    int x, y; // coordinate values
public:
    coord() { x=0; y=0; };
    coord(int i, int j) { x=i; y=j; }
    void get_xy(int &i, int &j) { i=x; j=y; }
    friend coord operator++(coord &ob);
```

```
};

// Overload ++ using a friend.
coord operator++(coord &ob) // use reference parameter
{
    ob.x++;
    ob.y++;

    return ob;
}

int main()
{
    coord o1(10, 10);
    int x, y;

    ++o1; // o1 is passed by reference
    o1.get_xy(x, y);
    cout << "(++o1) X: " << x << ", Y: " << y << "\n";

    return 0;
}
```

If you are using a modern compiler, you can also distinguish between the prefix and the postfix forms of the increment or decrement operators when using a friend operator function in much the same way you did when using member functions. You simply add an integer parameter when defining the postfix version. For example, here are the prototypes for both the prefix and postfix versions of the increment operator relative to the **coord** class:

```
coord operator++(coord &ob); // prefix
coord operator++(coord &ob, int notused); // postfix
```

If the ++ precedes its operand, the **operator++(coord &ob)** function is called. However, if the ++ follows its operand, the **operator++(coord &ob, int notused)** function is used. In this case, **notused** will be passed the value 0.

EXERCISES

1. Overload the **-** and **/** operators for the **coord** class using friend functions.
2. Overload the **coord** class so it can use **coord** objects in operations in which an integer value can be multiplied by each coordinate. Allow the operations to use either order: *ob * int* or *int * ob*.
3. Explain why the solution to Exercise 2 requires the use of friend operator functions.
4. Using a friend, show how to overload the – relative to the **coord** class. Define both the prefix and postfix forms.

6.6 A CLOSER LOOK AT THE ASSIGNMENT OPERATOR

As you have seen, it is possible to overload the assignment operator relative to a class. By default, when the assignment operator is applied to an object, a bitwise copy of the object on

the right is put into the object on the left. If this is what you want, there is no reason to provide your own **operator=()** function. However, there are cases in which a strict bitwise copy is not desirable. You saw some examples of this in Chapter 3, in cases in which an object allocates memory. In these types of situations, you will want to provide a special assignment operation.

EXAMPLES

1. Here is another version of the **strtype** class that you have seen in various forms in the preceding chapters. However, this version overloads the = operator so that the pointer **p** is not overwritten by an assignment operation.

```
#include <iostream>
#include <cstring>
#include <cstdlib>
using namespace std;

class strtype
{
    char *p;
    int len;
public:
    strtype(char *s);
    ~strtype()
    {
        cout << "Freeing " << (unsigned) p << '\n';
        delete [] p;
    }
    char *get() { return p; }
    strtype &operator=(strtype &ob);
};

strtype::strtype(char *s)
{
    int l;

    l = strlen(s)+1;

    p = new char [l];
    if(!p)
    {
        cout << "Allocation error\n";
        exit(1);
    }

    len = l;
    strcpy(p, s);
}

// Assign an object.
strtype &strtype::operator=(strtype &ob)
{
    // see if more memory is needed
```

```
    if(len < ob.len) // need to allocate more memory
    {
        delete []p;
        p = new char [ob.len];
        if(!p)
        {
            cout << "Allocation error\n";
            exit(1);
        }
    }
    len = ob.len;
    strcpy(p, ob.p);
    return *this;
}

int main()
{
    strtype a("Hello"), b("There");

    cout << a.get() << '\n';
    cout << b.get() << '\n';

    a = b; // now p is not overwritten

    cout << a.get() << '\n';
    cout << b.get() << '\n';

    return 0;
}
```

As you can see, the overloaded assignment operator prevents **p** from being overwritten. It first checks to see if the object on the left has allocated enough memory to hold the string that is being assigned to it. If it hasn't, that memory is freed and another portion is allocated. Then the string is copied to that memory and the length is copied into **len**.

Notice two other important features about the **operator=()** function. First, it takes a reference parameter. This prevents a copy of the object on the right side of the assignment from being made. As you know from previous chapters, when a copy of an object is made when passed to a function, that copy is destroyed when the function terminates. In this case, destroying the copy would call the destructor function, which would free **p**. However, this is the same **p** still needed by the object used as an argument. Using a reference parameter prevents this problem.

The second important feature of the **operator=()** function is that it returns a reference, not an object. The reason for this is the same as the reason it uses a reference parameter. When a function returns an object, a temporary object is created that is destroyed after the return is complete. However, this means that the temporary object's destructor will be called, causing **p** to be freed, but **p** (and the memory it points to) is still needed by the object being assigned a value. Therefore, by returning a reference, you prevent a temporary object from being created.

Note: *As you learned in Chapter 5, creating a coy constructor is another way to prevent both of the problems described in the preceding two paragraphs. But the copy constructor might not be as efficient a solution as using a reference parameter and a return reference*

type. This is because using a reference prevents the overhead associated with copying an object in either circumstances. As you can see, there are often several ways to accomplish the same end in C++. Learning to choose between them is part of becoming an excellent C++ programmer.

EXERCISES

1. Given the following class declaration, fill in all the details that will create a dynamic array type. That is, allocate memory for the array, storing a pointer to this memory in **p**. Store the size of the array, in bytes, in **size**. Have **put()** return a reference to the specified element, and have **get()** return the value of a specified element. Don't allow the boundaries of the array to be overrun. Also, overload the assignment operator so that the allocated memory of each array is not accidentally destroyed when one array is assigned to another. (In the next section you will see a way to improve your solution to this exercise.)

```
class dynarray
{
    int *p;
    int size;
public:
    dynarray(int s); // pass size of array in s
    int &put(int i); // return reference to element i
    int get(int i); // return value of element i
    // create operator=() function
};
```

6.7 OVERLOADING THE [ ] SUBSCRIPT OPERATOR

The last operator that we will overload is the [ ] array subscripting operator. In C++, the [ ] is considered a binary operator for the purposes of overloading. The [ ] can be overloaded only by a member function. Therefore, the general form of a member **operator[ ]()** function is as shown here:

```
type class_name::operator[](int index)
{
    // ...
}
```

Technically, the parameter does not have to be of type **int**, but **operator[ ]()** functions are typically used to provide array subscripting and as such an integer value is generally used.
To understand how the [ ] operator works, assume that an object called **O** is indexed as shown here:

```
O[9];
```

This index will translate into the following call to the **operator[ ]()** function:

```
O.operator[](9)
```

That is, the value of the expression within the subscripting operator is passed to the **operator[ ]()** function in its explicit parameter. The **this** pointer will point to **O**, the object that generated the call.

EXAMPLES

1. In the following program, **arraytype** declares an array of five integers. Its constructor function initializes each member of the array. The overloaded **operator**[ ]() function returns the value of the element specified by its parameter.

```
#include <iostream>
using namespace std;

const int SIZE = 5;

class arraytype
{
    int a[SIZE];
public:
    arraytype()
    {
        int i;
        for(i=0; i<SIZE; i++)
            a[i] = i;
    }
    int operator[](int i)
    {
        return a[i];
    }
};

int main()
{
    arraytype ob;
    int i;

    for(i=0; i<SIZE; i++)
        cout << ob[i] << " ";

    return 0;
}
```

This program displays the following output:

```
0 1 2 3 4
```

The initialization of the array **a** by the constructor in this and the following programs is for the sake of illustration only. It is not required.

2. It is possible to design the **operator**[ ]() function in such a way that the [ ] can be used on both the left and right sides of an assignment statement. To do this, return a reference to the element being indexed. For example, this program makes this change and illustrates its use:

```
#include <iostream>
using namespace std;

const int SIZE = 5;
```

```
class arraytype
{
    int a[SIZE];
public:
    arraytype()
    {
        int i;
        for(i=0; i<SIZE; i++)
            a[i] = i;
    }
    int &operator[](int i)
    {
        return a[i];
    }
};

int main()
{
    arraytype ob;
    int i;

    for(i=0; i<SIZE; i++)
        cout << ob[i] << " ";

    cout << "\n";

    // add 10 to each element in the array
    for(i=0; i<SIZE; i++)
        ob[i] = ob[i]+10; // [] on left of =

    for(i=0; i<SIZE; i++)
        cout << ob[i] << " ";

    return 0;
}
```

This program displays the following output:

```
0 1 2 3 4
10 11 12 13 14
```

Because the **operator[ ]()** function now returns a reference to they array element indexed by **i**, it can be used on the left side of an assignment to modify an element of the array. (Of course, it can still be used on the right side as well.) As you can see, this makes objects of **arraytype** act like normal arrays.

3. One advantage of being able to overload the [ ] operator is that it allows a better means of implementing safe array indexing. Earlier in this book you saw a simplified way to implement a safe array that relied upon functions such as **get()** and **put()** to access the elements of the array. Here you will see a better way to create a safe array that utilizes an overloaded [ ] operator. Recall that a safe array is an array that is encapsulated within a class that performs bounds checking. This approach prevents the array boundaries from being overrun. By overloading the [ ] operator for such an array, you allow it to be

accessed just like a regular array.

To create a safe array, simply add bounds checking to the **operator[ ]()** function. The **operator[ ]()** must also return a reference to the element being indexed. For example, this program adds a range check to the previous array program and proves that it works by generating a boundary error:

```
// A safe array example.
#include <iostream>
#include <cstdlib>
using namespace std;

const int SIZE = 5;

class arraytype
{
    int a[SIZE];
public:
    arraytype()
    {
        int i;
        for(i=0; i<SIZE; i++)
            a[i] = i;
    }
    int &operator[](int i);
};

// Provide range checking for arraytype.
int &arraytype::operator[](int i)
{
    if(i<0 || i>SIZE-1)
    {
        cout << "\nIndex value of ";
        cout << i << " is out of bounds.\n";
        exit(1);
    }
    return a[i];
}

int main()
{
    arraytype ob;
    int i;

    // this is OK
    for(i=0; i<SIZE; i++)
        cout << ob[i] << " ";

    /*
       this generates a run-time error because
       SIZE+100 is out of range
    */
    ob[SIZE+100] = 99; // error
```

```
    return 0;
}
```

In this program, when the statement

```
ob[SIZE+100] = 99;
```

executes, the boundary error is intercepted by **operator**[ ]() and the program is terminated before any damage can be done.

Because the overloading of the [ ] operator allows you to create safe arrays that look and act just like regular arrays, they can be seamlessly integrated into your programming environment. But be careful. A safe array adds overhead that might not be acceptable in all situations. In fact, the added overhead is why C++ does not perform boundary checking on array in the first place. However, in applications in which you want to be sure that a boundary error does not take place, a safe array will be worth the effort.

EXERCISES

1. ModifyExample 1 in Section 6.6 so that **strtype** overloads the [ ] operator. Have this operator return the character at the specified index. Also, allow the [ ] to be used on the left side of the assignment statement. Demonstrate its use.
2. Modify your answer to Exercise 1 from Section 6.6 so that it uses [ ] to index the dynamic array. That is, replace the **get()** and **put()** functions with the [ ] operator.

SKILLS CHECK

Mastery Skills Check

At this point you should be able to perform the following exercises and answer the questions.

1. Overload the >> and << shift operator relative to the **coord** class so that the following types of operations are allowed:

```
ob << integer
ob >> integer
```

Make sure your operations shift the **x** and **y** values by the amount specified.

2. Given the class

```
class three_d
{
    int x, y, z;
public:
    three_d(int i, int j, int k)
    {
        x = i; y = j; z = k;
    }
    three_d() { x=0; y=0; z=0; }
    void get(int &i, int &j, int &k)
    {
        i = x; j = y; k = z;
    }
};
```

overload the +, -, ++, and – operators for this class. (For the increment and decrement operators, overload only the prefix form.)

3. Rewrite your answer to Question 2 so that it uses reference parameters instead of value parameters to the operator functions. (Hint: You will need to use friend functions for the increment and decrement operators.)

4. How do friend operator functions differ from member operator functions?

5. Explain why you might need to overload the assignment operator.

6. Can **operator=()** be a friend function?

7. Overload the + for the **three_d** class in Question 2 so that it accepts the following types of operations:

```
ob + int;
int + ob;
```

8. Overload the ==, **!=**, and || operators relative to the **three_d** class from Question 2.

9. Explain the main reason for overloading the [ ] operator.

Cumulative Skills Check

This section checks how well you have integrated material in this chapter with that from the preceding chapters.

1. Create a **strtype** class that allows the following types of operators:

   - ➤ string concatenation using the + operator
   - ➤ string assignment using the = operator
   - ➤ string comparisons using <, >, and ==

   Feel free to use fixed-length strings. This is a challenging assignment, but with some thought (and experimentation), you should be able to accomplish it.